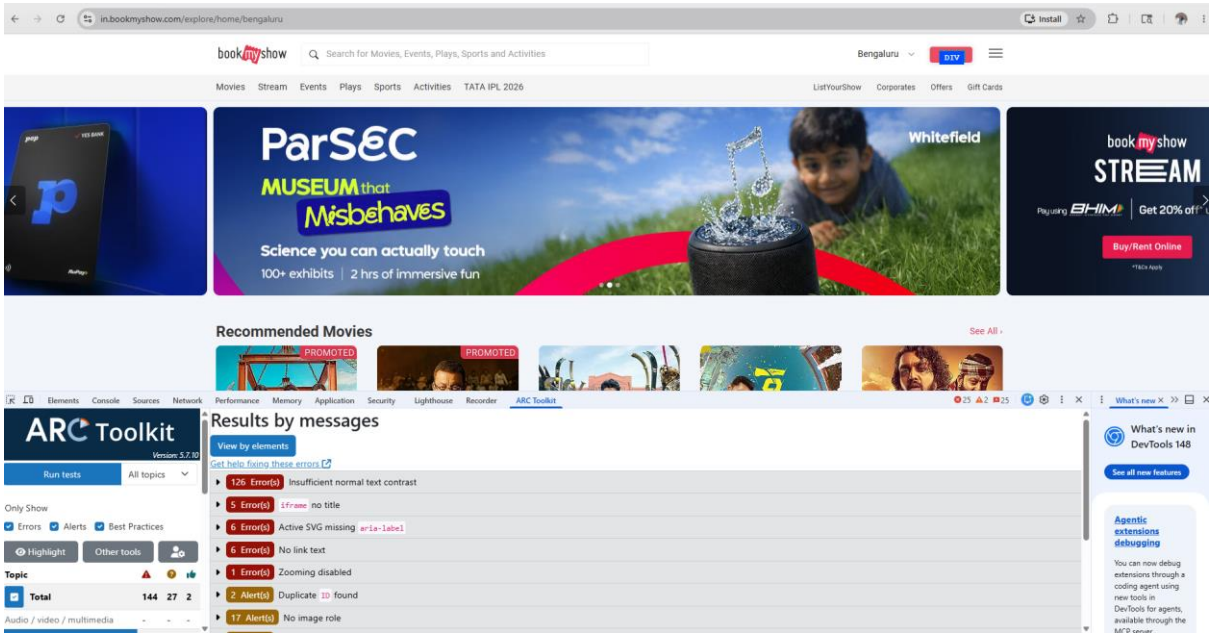
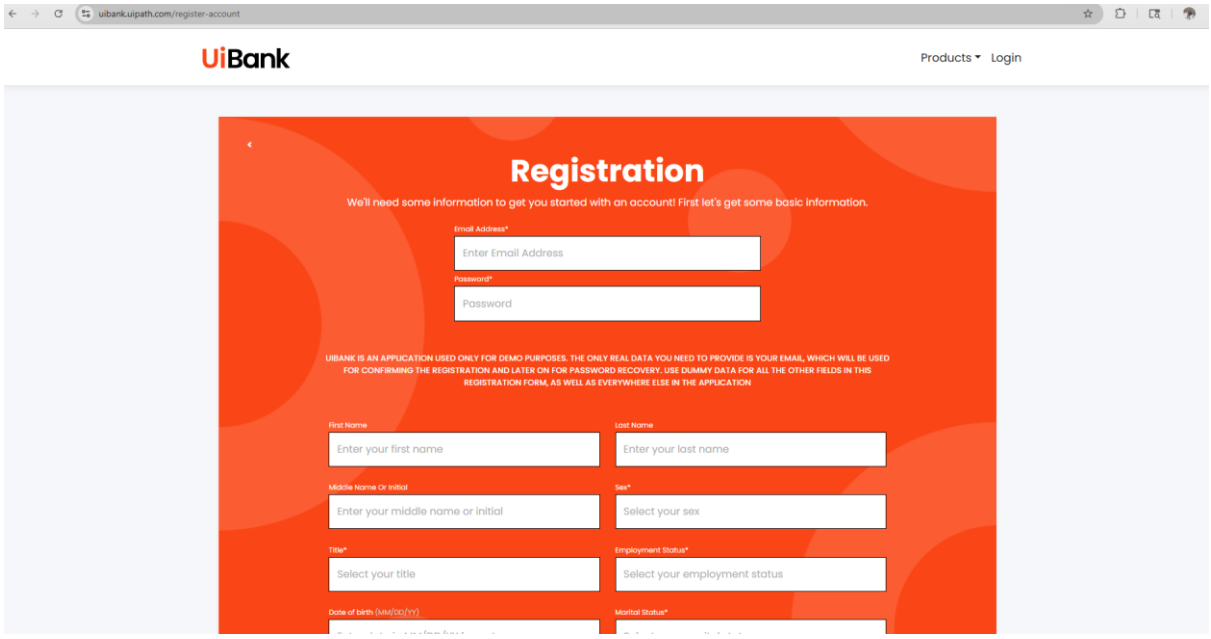
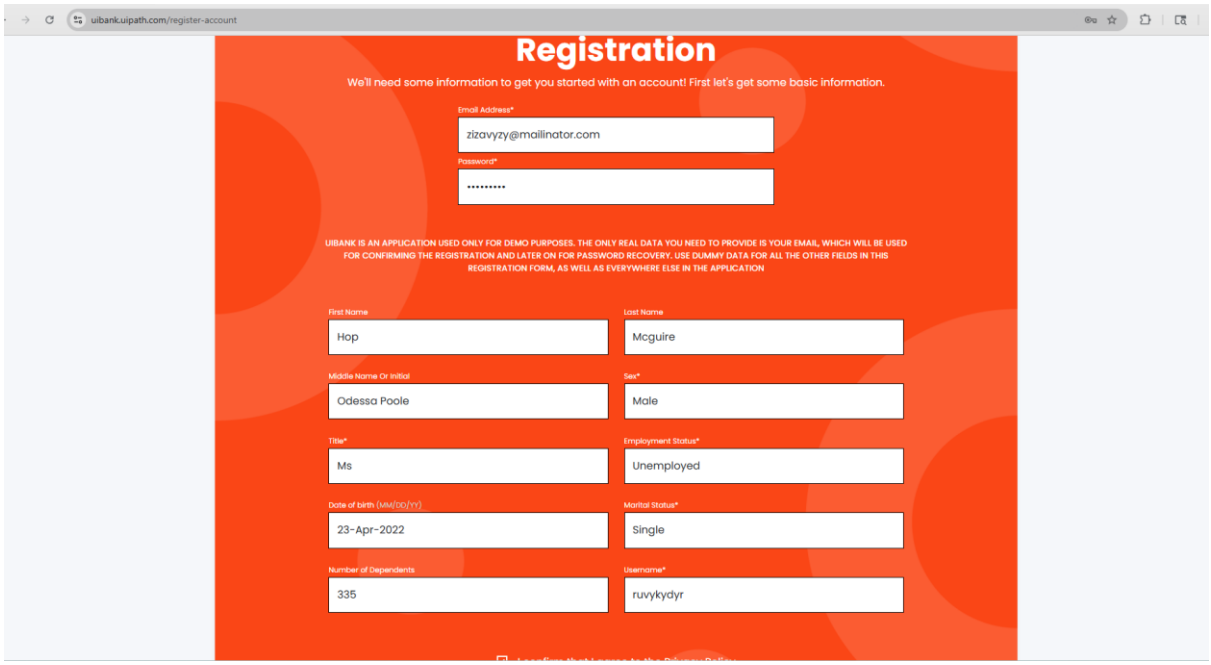
Week 3-day 1 Assignment of Adding Playwright + Typescript to Code Generator

Arc Toolkit Extension

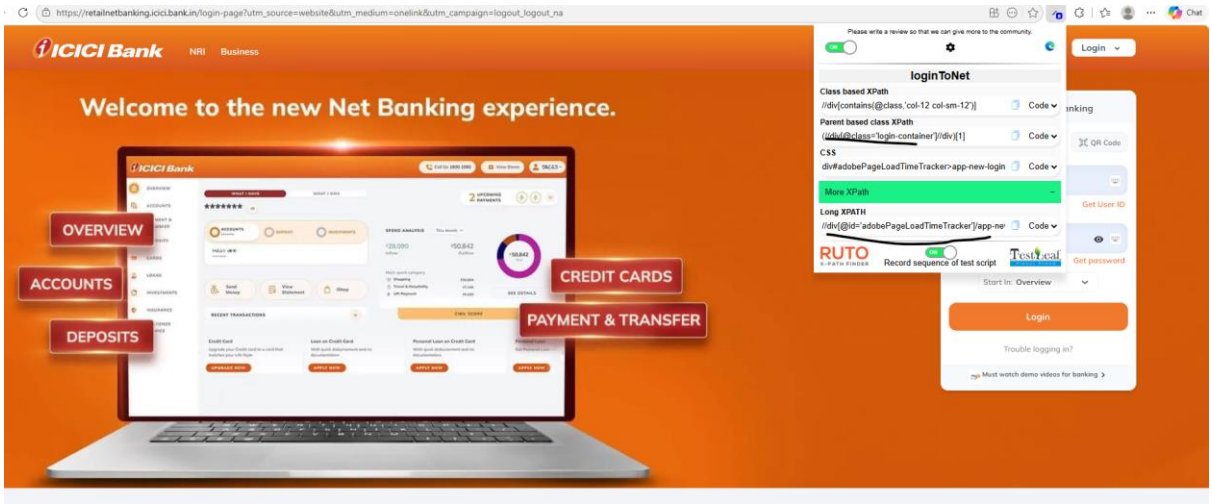


Fake Fill Extension



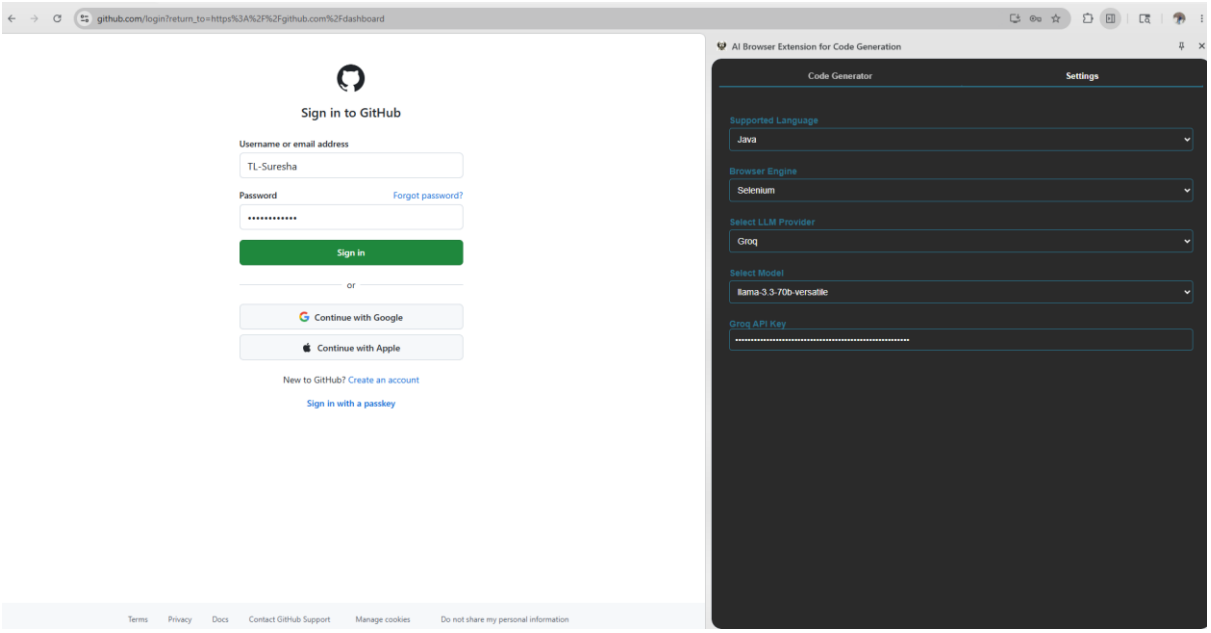


Ruto Path Finder



Vibe Coding for Playwright + TypeScript

Before adding Generated Code by using Java selenium



Output from code Generator:

```
Feature: Login to GitHub
  Scenario Outline: Successful login with valid credentials
    Given I open the GitHub login page
    When I type "<username>" into the Username field
    And I type "<password>" into the Password field
    And I click the Sign in button
    Then I should be logged in successfully

  Examples:
    | username        | password    |
    | "testuser123"    | "Test@123"  |
    | "seleniumuser"  | "Selenium@123" |
    | "cucumberuser"  | "Cucumber@123" |
```

```
package com.github.stepdefs;

import io.cucumber.java.en.*;
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;

import java.time.Duration;

public class GitHubLoginStepDefinitions {
```

```
    private WebDriver driver;
    private WebDriverWait wait;

    @io.cucumber.java.Before
    public void setUp() {
        driver = new ChromeDriver();
        wait = new WebDriverWait(driver, Duration.ofSeconds(10));
        driver.manage().window().maximize();
    }

    @io.cucumber.java.After
    public void tearDown() {
        if (driver != null) driver.quit();
    }

    @Given("I open the GitHub login page")
    public void openLoginPage() {
        driver.get("https://github.com/login?return_to=https%3A%2F%2Fgithub.com%2Fdashboard");
    }

    @When("I type {string} into the Username field")
    public void enterUsername(String username) {
        WebElement el = wait.until(ExpectedConditions.elementToBeClickable(By.id("login_field")));
        el.sendKeys(username);
    }

    @When("I type {string} into the Password field")
    public void enterPassword(String password) {
        WebElement el = wait.until(ExpectedConditions.elementToBeClickable(By.id("password")));
        el.sendKeys(password);
    }

    @When("I click the Sign in button")
    public void clickLogin() {
        driver.findElement(By.name("commit")).click();
    }

    @Then("I should be logged in successfully")
    public void verifyLogin() {
        // Add verification logic here
        // For example, you can check if a specific element is visible after login
        WebElement success =
wait.until(ExpectedConditions.visibilityOfElementLocated(By.className("header-nav-current-
user")));
        assert success.isDisplayed();
    }
}
```

Adding prompt to Github copilot to add playwright and Typescript

Read my entire workspace to understand and analyze how java selenium is implemented in the codebase and tell me what are all the impacted files.
I want to implement the playwright and typescript same like selenium java

Chat output from Github Copilot

Optimized tool selection
I'll analyze your workspace to understand the Java Selenium implementation and identify all impacted files.
Now let me create a memory file with this analysis and prepare a comprehensive summary:
Based on my thorough analysis of your workspace, here's what I found:
Java Selenium Implementation - Complete Analysis
All Impacted Files:

File	Purpose	Key Changes Needed for Playwright TS
panel.html	Settings UI with language/engine selectors	Already has TypeScript + Playwright options; checkbox names need updating
chat.js	**MAIN** - Orchestration logic for code generation	Add TypeScript+Playwright detection; update getPromptKeys() logic
prompts.js	**CRITICAL** - Prompt templates for LLM	Add Playwright TypeScript prompt templates
groq-api.js	LLM provider integration	No changes needed
openai-api.js	LLM provider integration	No changes needed
testleaf-api.js	LLM provider integration	No changes needed
content.js	DOM element inspector	No changes needed
tabs.js	UI tab switching	No changes needed
bg.js	Background messaging	No changes needed
popup.js	Settings panel logic	May need language-specific checkbox labels

---

How Java Selenium is Implemented:
1. Detection & Routing (chat.js)

- Helper methods check language + engine: isJavaSelenium(language, engine)
- Only java + selenium combination is fully supported
- Other combinations trigger "not yet supported" message

2. Prompt Selection (chat.js - getPromptKeys())

-
-
-
-

3. Prompt Templates (prompts.js)

- SELENIUM_JAVA_PAGE_ONLY: Generates Java Page Object classes with WebDriver/Selenium imports

- CUCUMBER_WITH_SELENIUM_JAVA_STEPS: Generates .feature file + Java step definitions with WebDriver setup, explicit waits, By selectors
- CUCUMBER_ONLY: Gherkin-only feature files

4. Code Generation Flow

- User selects: Java + Selenium + DOM elements
- Selects generation modes (Feature, Page, or both)
- LLM generates code based on selected prompt template
- Displays with syntax highlighting

Implementation Status:

✅ **Java + Selenium** - Fully implemented

⚠️ **TypeScript + Playwright** - Options exist in UI but NOT implemented (shows "not yet supported")

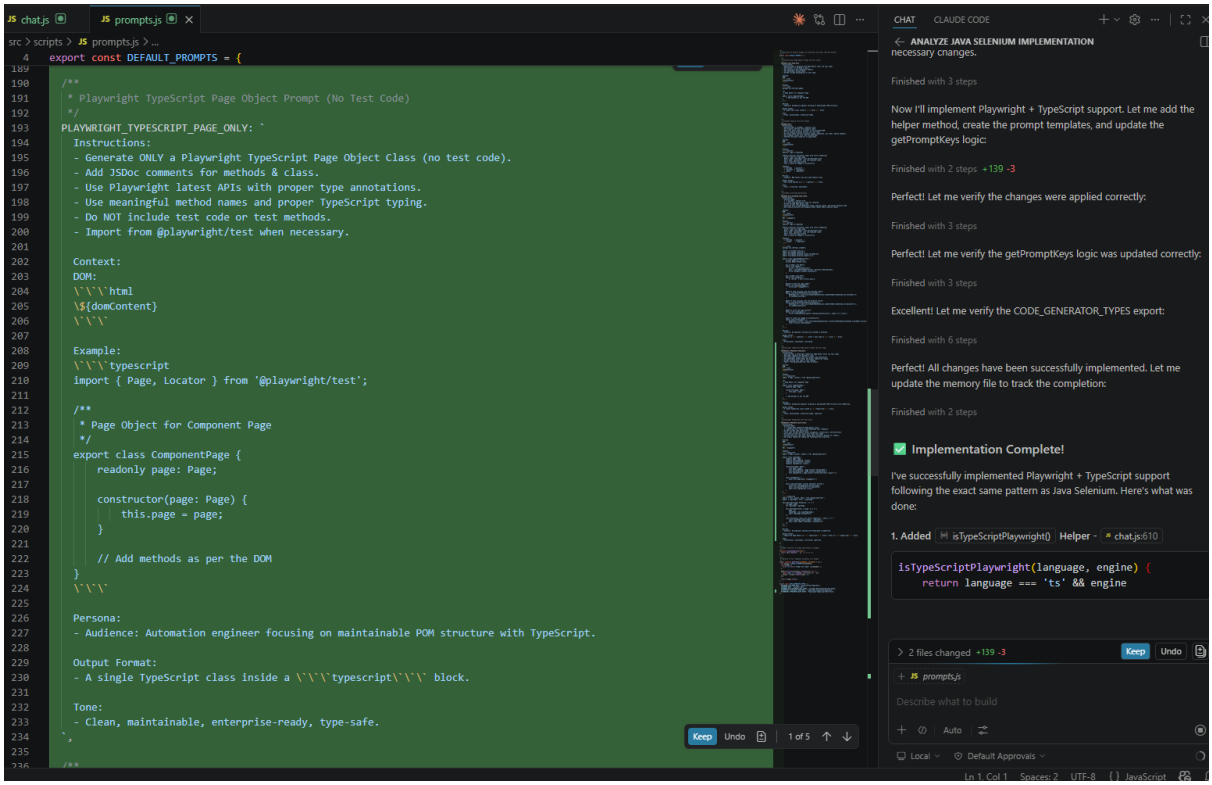
❌ **All other combinations** - Not implemented

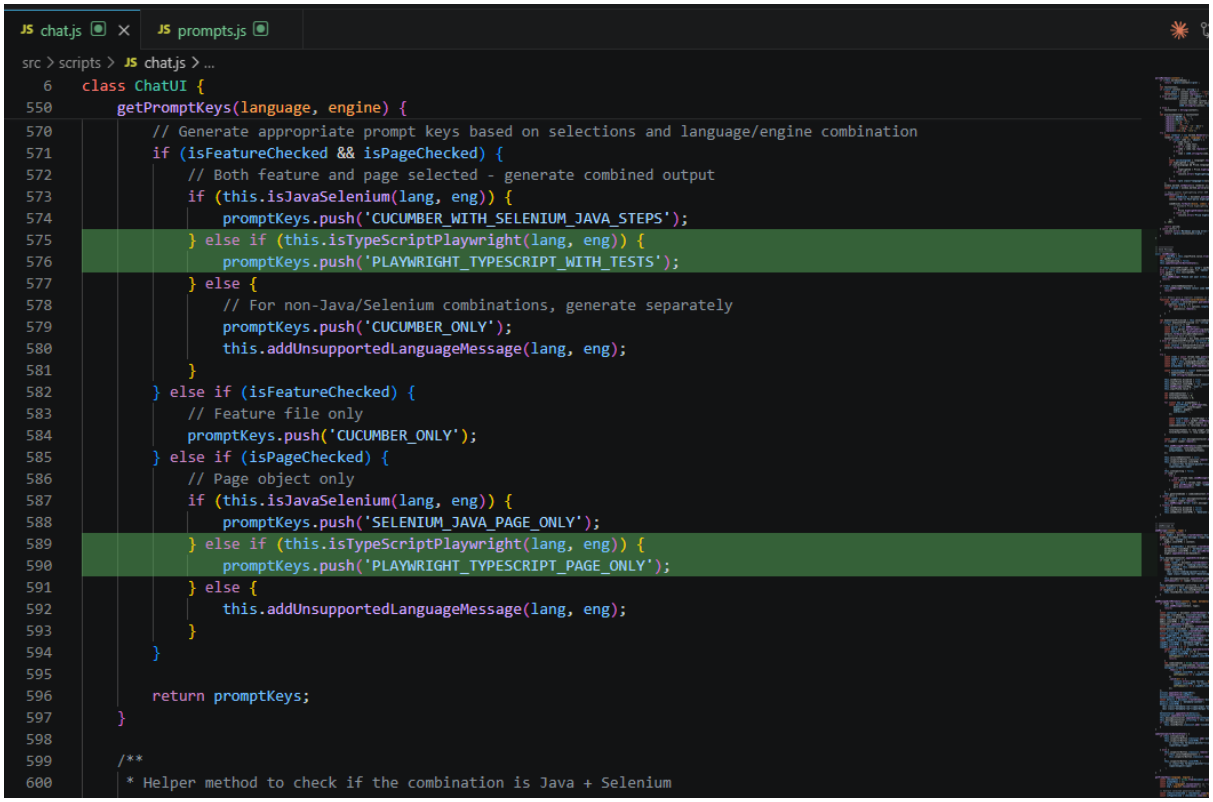
To Implement Playwright + TypeScript (Similar Pattern):

You'll need to:

1. Add isTypeScriptPlaywright() helper in chat.js
2. Create 2-3 new prompt templates in prompts.js:
   - PLAYWRIGHT_TYPESCRIPT_PAGE_ONLY
   - PLAYWRIGHT_TYPESCRIPT_WITH_TESTS
3. Update getPromptKeys() logic to handle TypeScript + Playwright
4. Optionally rename "javaGenerationMode" to generic "generationMode"
5. Update loading message to show "Playwright" when TypeScript + Playwright is selected

Would you like me to proceed with implementing the Playwright + TypeScript prompts and logic following the exact same pattern as Java Selenium?

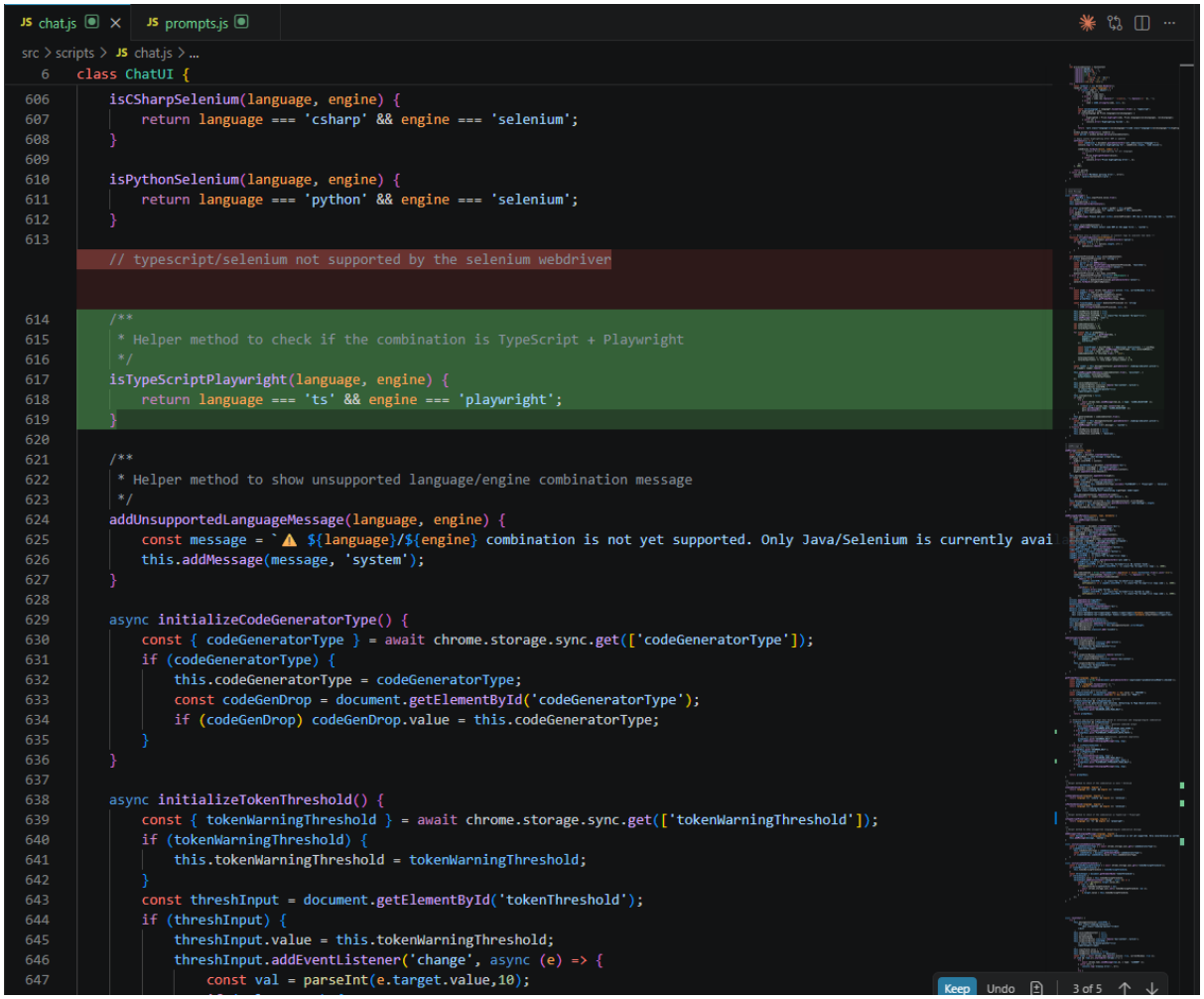




chat.js | prompts.js

src > scripts > chat.js > ...

```
6   class ChatUI {
550     getPromptKeys(language, engine) {
570         // Generate appropriate prompt keys based on selections and language/engine combination
571         if (isFeatureChecked && isPageChecked) {
572             // Both feature and page selected - generate combined output
573             if (this.isJavaSelenium(lang, eng)) {
574                 promptKeys.push('CUCUMBER_WITH_SELENIUM_JAVA_STEPS');
575             } else if (this.isTypeScriptPlaywright(lang, eng)) {
576                 promptKeys.push('PLAYWRIGHT_TYPESCRIPT_WITH_TESTS');
577             } else {
578                 // For non-Java/Selenium combinations, generate separately
579                 promptKeys.push('CUCUMBER_ONLY');
580                 this.addUnsupportedLanguageMessage(lang, eng);
581             }
582         } else if (isFeatureChecked) {
583             // Feature file only
584             promptKeys.push('CUCUMBER_ONLY');
585         } else if (isPageChecked) {
586             // Page object only
587             if (this.isJavaSelenium(lang, eng)) {
588                 promptKeys.push('SELENIUM_JAVA_PAGE_ONLY');
589             } else if (this.isTypeScriptPlaywright(lang, eng)) {
590                 promptKeys.push('PLAYWRIGHT_TYPESCRIPT_PAGE_ONLY');
591             } else {
592                 this.addUnsupportedLanguageMessage(lang, eng);
593             }
594         }
595
596         return promptKeys;
597     }
598
599     /**
600      * Helper method to check if the combination is Java + Selenium
```



chat.js | prompts.js

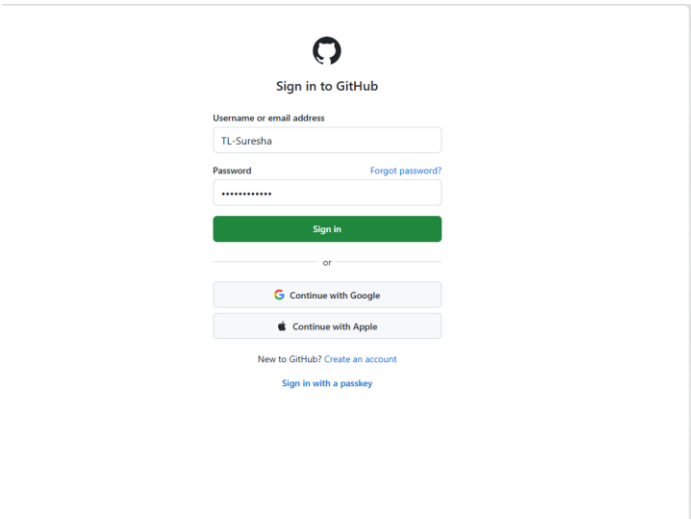
src > scripts > chat.js > ...

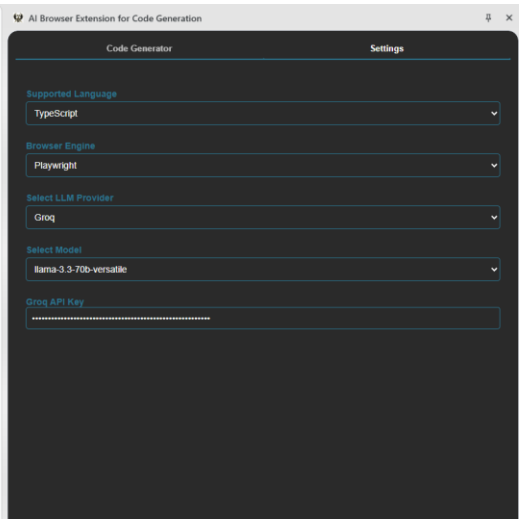
```
6   class ChatUI {
606     isCSharpSelenium(language, engine) {
607         return language === 'csharp' && engine === 'selenium';
608     }
609
610     isPythonSelenium(language, engine) {
611         return language === 'python' && engine === 'selenium';
612     }
613
        // typescript/selenium not supported by the selenium webdriver


614     /**
615      * Helper method to check if the combination is TypeScript + Playwright
616      */
617     isTypeScriptPlaywright(language, engine) {
618         return language === 'ts' && engine === 'playwright';
619     }
620
621     /**
622      * Helper method to show unsupported language/engine combination message
623      */
624     addUnsupportedLanguageMessage(language, engine) {
625         const message = `⚠️ ${language}/${engine} combination is not yet supported. Only Java/Selenium is currently avai
626         this.addMessage(message, 'system');
627     }
628
629     async initializeCodeGeneratorType() {
630         const { codeGeneratorType } = await chrome.storage.sync.get(['codeGeneratorType']);
631         if (codeGeneratorType) {
632             this.codeGeneratorType = codeGeneratorType;
633             const codeGenDrop = document.getElementById('codeGeneratorType');
634             if (codeGenDrop) codeGenDrop.value = this.codeGeneratorType;
635         }
636     }
637
638     async initializeTokenThreshold() {
639         const { tokenWarningThreshold } = await chrome.storage.sync.get(['tokenWarningThreshold']);
640         if (tokenWarningThreshold) {
641             this.tokenWarningThreshold = tokenWarningThreshold;
642         }
643         const threshInput = document.getElementById('tokenThreshold');
644         if (threshInput) {
645             threshInput.value = this.tokenWarningThreshold;
646             threshInput.addEventListener('change', async (e) => {
647                 const val = parseInt(e.target.value,10);
```

Keep Undo 3 of 5

After updating the Codebase

In AI Generator





Output from code generator of test case is

```
 import { Page, Locator, expect } from '@playwright/test';

export class LoginPage {
   readonly page: Page;
   readonly usernameField: Locator;
   readonly passwordField: Locator;
   readonly loginButton: Locator;

   constructor(page: Page) {
      this.page = page;
      this.usernameField = page.locator('#login_field');
      this.passwordField = page.locator('#password');
      this.loginButton = page.locator('input:has-text("Sign in")');
   }

   async navigateTo() {
      await
this.page.goto('https://github.com/login?return_to=https%3A%2F%2Fgithub.com%2Fdashboard');
   }

   async login(username: string, password: string) {
      await this.usernameField.fill(username);
      await this.passwordField.fill(password);
      await this.loginButton.click();
   }
}

import { test, expect, Page } from '@playwright/test';
import { LoginPage } from './LoginPage';

test.describe('Login Scenarios', () => {
```

```
  let page: Page;
  let loginPage: LoginPage;

  test.beforeEach(async ({ page: p }) => {
    page = p;
    loginPage = new LoginPage(page);
    await loginPage.navigateTo();
  });

  test('Successful login with valid credentials', async () => {
    await loginPage.login('testuser', 'testpassword123');
    await expect(page).toHaveURL(/.*dashboard/);
  });

  test('Unsuccessful login with invalid credentials', async () => {
    await loginPage.login('invaliduser', 'invalidpassword');
    await expect(page).not.toHaveURL(/.*dashboard/);
  });

  test('Login with empty username', async () => {
    await loginPage.login('', 'testpassword123');
    await expect(loginPage.usernameField).toBeFocused();
  });

  test('Login with empty password', async () => {
    await loginPage.login('testuser', '');
    await expect(loginPage.passwordField).toBeFocused();
  });

  test('Login with valid South Indian username and password', async () => {
    await loginPage.login('rameshwaran', 'tamilnadu123');
    await expect(page).toHaveURL(/.*dashboard/);
  });
});
```